CSC490A5 - Group 8

Isaac Abell - 1008831329, Thomas(Yehyun) Lee - 1008992217,
Zeling (Zoey) Zhang - 1007714779, Jessica Zhang- 1008494130

Table of Contents

Part One: Profiling Execution Time..... 1
Part Two: Showing Code Coverage..... 4
A5 Test Coverage Report..... 4
Web App Test Coverage Report..... 6
Part Three: Test Coverage Breadth.....8
Part Five: Two Memorable Bugs..... 12
Bug 1 Alignment algorithm worked in practice but failed on the testing set.....12
Bug1Video..... 12
Bug 2. Misalignment in OS - Windows: FFmpeg discovery / EBS pipeline blocked..... 13

Part One: Profiling Execution time

For our project, our active project scope is currently `A5/` + `web-app/` folders. The web app calls the A5 FastAPI service directly. Earlier in work of A5, we previously relied on backend code in A2 folder, but we now have migrated those to A5 folder and we no longer depend on `A2/` to run the product flow. A5 contains backend FastAPI service Python code and web-app folder contains Next.js monolith code which also contains some backend logic in TypeScript.

The profiling code could be run with "./A5/venv/bin/python A5/profiling/find_bottlenecks_alignment_segment.py" command.

This script mirrors the same major pipeline that the web app triggers through the A5 API:
1) Extract audio and load waveforms
2) Compute chroma features
3) Align reference vs practice audio
4) Beat tracking on the shared window
5) Generate beat-synced segments (8-beat chunks)

The profiled `src/` steps are `load_audio_files`, `extract_chroma`, `perform_alignment`, `track_beats`, and `generate_segments`. The profiling script writes the bar chart to `A5/profiling/profile_results.png`.

After profiling five core functions within our alignment and segmentation algorithms (the slowest and most important step for end-to-end user experience) using Python's `profile` module (see chart below), we targeted the two primary bottlenecks:

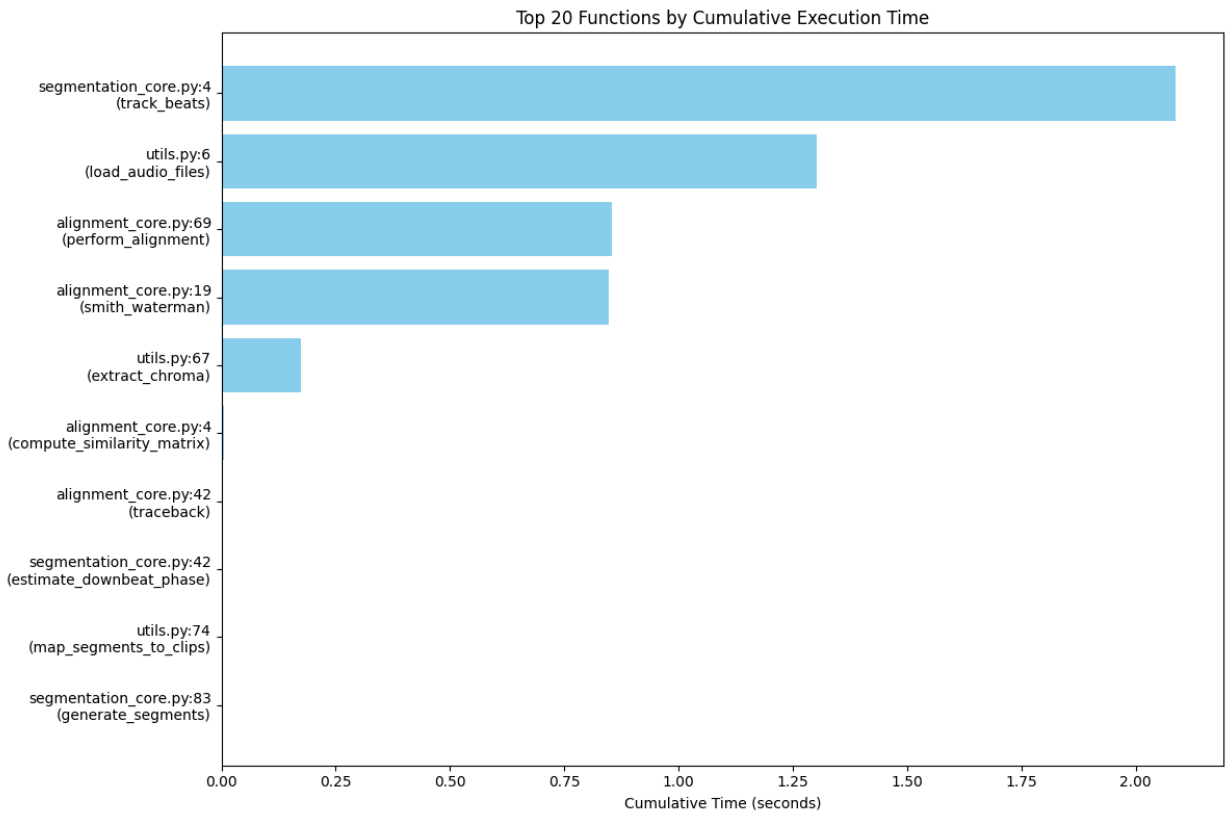

Figure 1. Showing the bottleneck before the improvement

- **`smith_waterman` (Alignment):** Replaced nested `for` loops with vectorized operations, resulting in the significant runtime reduction shown in the graph.
- **`load_audio_files` (Segmentation):** Implemented concurrent loading for images and `librosa` audio. This provided a very slight speedup, though not the 2x improvement you would expect due to Python's (GIL) restricting true multithreading.

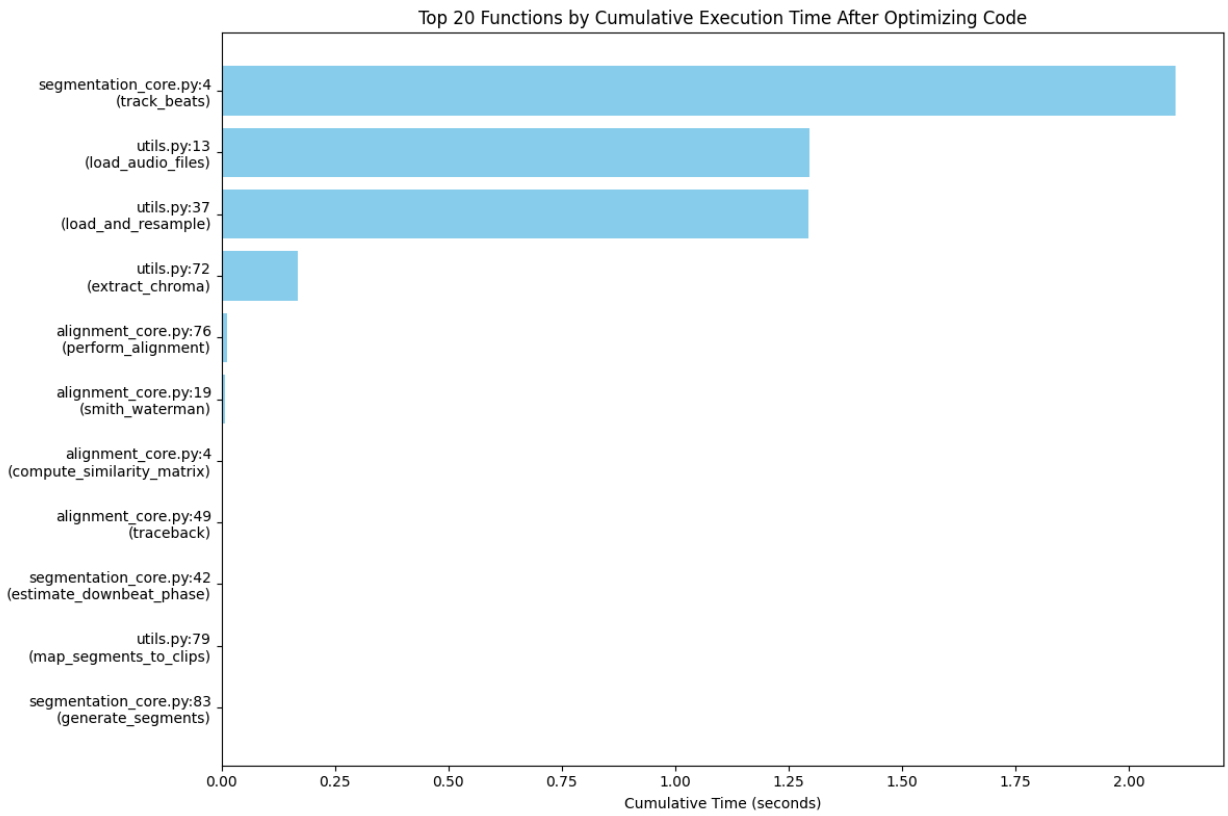

Figure 2. Showing the improvement

Part Two: Showing Code Coverage

Our implementation (we use pytest-cov for A5 and Vitest's v8 reporter for web-app; CI updates the markdown snapshots and posts coverage on PRs):

Coverage is automated for both backend and frontend side (`A5/` and `web-app/`) through CI. Because the project includes two stacks with Python and TypeScript, we report coverage separately and automate both.

Local commands:

```
# A5 backend
cd A5
pytest --cov=src -s

# web app
cd ../web-app
npm run coverage
```

Single-command local test run:
"./scripts/test_all.sh"

GitHub Actions workflow:
- Workflow file: `.github/workflows/a5-tests-and-code-coverage.yml`
- Runs on: PR/push for `main` and `a5` branches when `A5/`, `web-app/`, or `scripts/` change
- Outputs:
  - PR coverage comments for A5 and web-app
  - committed snapshots:
    - `A5/test_coverage.md`:

A5 Test Coverage Report

Last Updated: Fri Mar 27 21:42:44 UTC 2026

```
==================== test session starts ====================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\zsq59\csc490\TempoFlow\a5
plugins: anyio-4.13.0, cov-7.1.0
collected 187 items

tests\alignment_and_segmentation\test_alignment_algo_accuracy.py No data source found (S3 or Local). Skipping.
s
tests\alignment_and_segmentation\test_alignment_core.py .... .
...........
tests\alignment_and_segmentation\test_router.py .........
```

```
tests\alignment_and_segmentation\test_segmentation_core.py .....
tests\alignment_and_segmentation\test_utils.py .....
tests\test_ebs_web_adapter.py .....
tests\test_ffmpeg_paths.py .....
tests\test_gemini_move_feedback.py .....tests\test_main.py .....
tests\test_overlay_api.py .....

====================== tests coverage ======================
_____ coverage: platform win32, python 3.11.9-final-0 _______

Name                                                Stmts
 Miss  Cover
----------------------------------------------------------------------
src\__init__.py                                         0      0   100%
src\alignment_and_segmentation\__init__.py              4      0   100%
src\alignment_and_segmentation\alignment_core.py       29      0   100%
src\alignment_and_segmentation\router.py               62      0   100%
src\alignment_and_segmentation\schemas.py              17      0   100%
src\alignment_and_segmentation\segmentation_core.py    59      0   100%
src\alignment_and_segmentation\utils.py                58      0   100%
src\ebs_web_adapter.py                                213      0   100%
src\ffmpeg_paths.py                                    57      0   100%
src\gemini_move_feedback.py                           192      0   100%
src\main.py                                           153      0   100%
src\overlay_api.py                                    474     68    86%
----------------------------------------------------------------------
TOTAL                                                1318     68    95%
============= 186 passed, 1 skipped in 18.56s ==============
```
(image view below)

Coverage 95%

▼A5 Code Coverage Report

File	Stmts	Miss	Cover	Missing
src				
__init__.py	0	0	100%	
ebs_web_adapter.py	213	0	100%	
ffmpeg_paths.py	57	0	100%	
gemini_move_feedback.py	192	0	100%	
main.py	153	0	100%	
overlay_api.py	474	68	68	86%
src/alignment_and_segmentation				
__init__.py	4	0	100%	
alignment_core.py	29	0	100%	
router.py	62	0	100%	
schemas.py	17	0	100%	
segmentation_core.py	59	0	100%	
utils.py	58	0	100%	
TOTAL	**1318**	**68**	**95%**	

Web App Test Coverage Report

Last Updated: Fri Mar 27 21:37:49 UTC 2026

```
(illegible chars omitted)
> web-app@0.1.0 coverage
> vitest run --coverage
-------------------|---------|----------|---------|---------|-------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered
Line #s
-------------------|---------|----------|---------|---------|-------------
All files          |   56.32 |    42.14 |   61.26 |   57.63 |
 app               |     100 |      100 |     100 |     100 |
  layout.tsx       |     100 |      100 |     100 |     100 |
  page.tsx         |     100 |      100 |     100 |     100 |
 app/analysis      |   31.85 |    18.54 |    37.5 |   31.73 |
  page.tsx         |   31.85 |    18.54 |    37.5 |   31.73 |
...94-497,514-613
```

```
 app/api/coach      |   96.66 |    85.71 |     100 |   96.66 |
  route.ts          |   96.66 |    85.71 |     100 |   96.66 | 72
 ...-pose-feedback |   82.66 |    66.66 |   88.88 |    85.5 |
  route.ts          |   82.66 |    66.66 |   88.88 |    85.5 |
...36,179,185-186
 ...pi/init-webrtc |   83.33 |    85.71 |     100 |   83.33 |
  route.ts          |   83.33 |    85.71 |     100 |   83.33 | 72-74
 app/api/process    |   96.87 |    77.77 |     100 |   96.87 |
  route.ts          |   96.87 |    77.77 |     100 |   96.87 | 24
 app/api/upload     |   88.23 |    78.57 |     100 |   88.23 |
  route.ts          |   88.23 |    78.57 |     100 |   88.23 | 57-58
 app/dashboard      |     100 |       70 |     100 |     100 |
  page.tsx          |     100 |       70 |     100 |     100 | 80,87
 app/ebs-viewer     |     100 |      100 |     100 |     100 |
  page.tsx          |     100 |      100 |     100 |     100 |
 app/upload         |   28.16 |    23.07 |   29.62 |   28.65 |
  page.tsx          |   28.16 |    23.07 |   29.62 |   28.65 |
...55-300,310-311
 components         |   34.79 |     21.5 |   48.21 |   36.01 |
  ...ixOverlay.tsx |   36.55 |    30.61 |   71.42 |    37.5 | 66-149
  PoseOverlay.tsx   |   20.61 |    12.05 |      45 |   21.28 |
...13-337,353-387
  ...meOverlay.tsx |    31.3 |    15.06 |   33.33 |   32.69 |
...3,50-52,69-162
  ...eoOverlay.tsx |   54.41 |     9.09 |   46.66 |    62.5 |
...68,71-73,78,81
  ...veOverlay.tsx |   37.28 |    22.37 |   47.22 |    38.3 |
...94,298-304,371
  ...eoOverlay.tsx |   94.87 |    86.36 |      80 |   94.44 | 74-75
  ...ntOverlay.tsx |   33.33 |    27.58 |   47.05 |   35.16 |
...47-287,309-347
 components/ebs     |   37.27 |    32.32 |   35.23 |   38.76 |
  ...nceViewer.tsx |   84.78 |    92.85 |   66.66 |    87.8 |
46,57,63-64,94
  EbsViewer.tsx     |       0 |        0 |       0 |       0 | 64-2241
  ...ckOverlay.tsx |   22.27 |    16.34 |   38.09 |   23.49 |
...25-239,257-355
  ...backPanel.tsx |   81.98 |     73.1 |    82.6 |      85 |
...72,211,318-336
  ...ackViewer.tsx |   61.47 |    43.72 |   44.44 |   64.84 |
...03-604,641-838
  ...backPanel.tsx |   85.82 |     68.3 |   81.81 |   89.56 |
...55,203,310-331
  ...iewerLogic.ts |   81.81 |    73.52 |   83.33 |   93.33 | 7-8
  types.ts          |       0 |        0 |       0 |       0 |
  useEbsViewer.ts   |   56.37 |    36.69 |   42.55 |    61.5 |
...68,475-479,509
```

```
 .../ebs/__mocks__ |      70 |      100 |      40 |     100 |
  ...iewerLogic.ts |      70 |      100 |      40 |     100 |
 lib               |   87.56 |    73.92 |   87.16 |   90.99 |
  analysis.ts      |     100 |    68.08 |     100 |     100 |
...29-181,191-199
  ...Comparison.ts |       0 |        0 |       0 |       0 |
  ...yGenerator.ts |    93.4 |    73.33 |    90.9 |   96.47 | 62-63,114
  ebsStorage.ts    |    90.9 |       75 |   83.33 |     100 | 16
  ...emporalLlm.ts |     100 |      100 |     100 |     100 |
  ...yGenerator.ts |   94.07 |    67.64 |   93.75 |   98.38 | 44-45
  ...yGenerator.ts |   89.16 |    73.46 |   85.71 |   89.65 |
14-15,38,43,91-98
  normalization.ts |   70.58 |    61.11 |   83.33 |   70.96 | 100-113
  ...aySegments.ts |      80 |    67.08 |      85 |   88.37 |
31,60,82,112,136
  ...layStorage.ts |   91.17 |    83.33 |   83.33 |     100 | 58
  poseAnalysis.ts  |   94.68 |    73.33 |      92 |    97.5 | 55-56
  ...layStorage.ts |   93.47 |    70.83 |   80.76 |     100 |
25-86,121,133,152
  ...ionStorage.ts |      88 |    73.33 |     100 |   97.56 | 108
  videoStorage.ts  |    92.1 |       75 |   86.95 |     100 | 16
  ...yGenerator.ts |    66.9 |    55.55 |   70.96 |   71.25 |
...18,428-429,439
  ...layStorage.ts |   89.87 |       60 |   78.26 |   96.87 | 104-105
 lib/bodyPix       |     100 |    97.08 |     100 |     100 |
  beatFeedback.ts  |     100 |     92.5 |     100 |     100 | 197-215
  compare.ts       |     100 |      100 |     100 |     100 |
  constants.ts     |     100 |      100 |     100 |     100 |
  feedbackCopy.ts  |     100 |      100 |     100 |     100 |
  geometry.ts      |     100 |      100 |     100 |     100 |
  index.ts         |       0 |        0 |       0 |       0 |
  ...onFeatures.ts |     100 |      100 |     100 |     100 |
  segmentation.ts  |     100 |      100 |     100 |     100 |
  stats.ts         |     100 |      100 |     100 |     100 |
  timestamps.ts    |     100 |      100 |     100 |     100 |
  types.ts         |     100 |      100 |     100 |     100 |
-------------------|---------|----------|---------|---------|-------------
```

Part Three: Test Coverage Breadth

The percentages below are the ones we report from the Part Two coverage setup.

Current coverage snapshot:
- A5 backend: 95% total statement coverage (`pytest --cov=src`)
- web-app: 56.32% total statement coverage (`vitest --coverage`, v8 “All files” row; line coverage ≈57.63%)

As we will mention below, we have code for YOLO, MoveNet, FastSAM, Sam3, Roboflow models and UI/UX that we have added for experimentation purpose. Those will be removed for final project but in a5 branch, they remain as is, and won’t be fully covered for testing as they will be removed.

Breadth currently covered well:

- A5 alignment/segmentation core + web adapter + API routes (contract tests for request/response shapes)
- A5 overlay endpoints and worker error-handling branches (model runtimes mocked where necessary)
- Web app API routes that are core to the flow: `/api/process`, `/api/coach`, `/api/ebs-pose-feedback`, `/api/upload`
- Web app “core math/logic” utilities: normalization (`calculateAlignmentTransform`), storage (IndexedDB helpers), and BodyPix comparison core (`src/lib/bodyPix/*`)
- Overlay playback infrastructure components that don’t require real ML inference: `PrecomputedFrameOverlay`, `PrecomputedVideoOverlay`, `ProgressiveOverlay`, `DifferenceViewer`

Remaining gap:

- Web app UI breadth is still lower than backend because the largest files are UI-heavy and depend on browser-only primitives (video elements, canvas contexts, WebGL). Notably:
  - `app/analysis/page.tsx` (large UI page)
  - `components/ebs/EbsViewer.tsx` (very large orchestrator component)
  - `components/ebs/useEbsViewer.ts` (hook with DOM/video timing)
- These are being addressed by extracting pure logic into testable helper modules (e.g., `components/ebs/ebsViewerLogic.ts`), then unit-testing that logic thoroughly.

Scope justification:

- Our main product path is BodyPix + normalization + EBS web-app flow.
- As mentioned, we keep YOLO/MoveNet/FastSAM/Sam3/Roboflow in the repository as benchmarking options for the final project, but they are not the primary user path and are not required for the A5 scope. We have explicitly prioritize coverage for the core user flow and its failure modes.

Part Four: Test Coverage Depth

We chose the audio alignment module as our “depth” target because it is both:

1. the most complex algorithmic component (dynamic programming + signal similarity), and
2. the highest-impact correctness dependency (bad alignment breaks segment mapping, overlays, and feedback).

Target module: A5/src/alignment_and_segmentation/alignment_core.py

Where the depth tests live: A5/tests/alignment_and_segmentation/test_alignment_core.py

In this test file, we have organized tests into:

- **positive tests** (core expected behaviour),
- **edge cases** (unusual but valid inputs), and
- **negative tests** (failure modes / invalid inputs).

For each test, we have included an inline **WHY** comment explaining the specific bug class or failure mode it prevents per the Part 4 requirement.

Positive / core behaviour

1. test_compute_similarity_matrix_identical
    - Verifies the cosine similarity foundation returns 1.0 for identical frames.
2. test_compute_similarity_matrix_orthogonal
    - Ensures truly mismatched frames return 0.0 (prevents false-positive alignment).
3. test_smith_waterman_perfect_match
    - Checks DP recurrence accumulation along the diagonal on a perfect match.
4. test_smith_waterman_offset
    - Validates alignment still works when the match occurs at an offset (realistic “snippet” case).
5. test_smith_waterman_all_ones
    - Stress-tests DP accumulation when all entries match (catches recurrence/index bugs).
6. test_traceback_standard_path
    - Ensures traceback returns correct 0-indexed bounds for the aligned region.
7. test_perform_alignment_integration
    - Integration test across similarity → DP → traceback to confirm the end-to-end contract.
8. test_rectangular_matrices
    - Handles different clip lengths (common source of swapped-dimension / shape bugs).
9. test_integration_partial_overlap

    - Real-world partial overlap (clip B appears inside clip A); ensures correct boundaries.

Edge cases (valid but tricky)

10. test_smith_waterman_no_match
    - Ensures “no match” produces an all-zero DP matrix (prevents hallucinated paths).
11. test_traceback_short_path
    - Single-cell match; ensures traceback loop terminates safely without OOB reads.
12. test_traceback_zero_matrix
    - Completely zero matrix; ensures the function returns sentinel indices without crashing.
13. test_smith_waterman_empty_input
    - Empty similarity matrix; ensures the function returns a safe 1×1 output.

Negative tests (failure modes)

14. test_empty_chroma_input
    - Corrupted inputs (0-length chroma) should raise, not silently align.
15. test_dimension_mismatch
    - Rejects incompatible chroma bin dimensions (prevents nonsensical similarity scores).
16. test_nan_values_in_chroma
    - Rejects NaNs (prevents silent propagation and unpredictable DP/traceback outputs).

Why this module for depth:

- It’s the “root of trust” for the entire TempoFlow pipeline: a small error can create large UX failures (wrong segments, wrong beat mapping, overlays misaligned).
- The algorithm has multiple interacting steps and common failure modes (shape mismatches, empty inputs, NaNs, no-match degeneracy, offsets), making it a strong candidate for depth testing.

Part Five: Two Memorable Bugs

Record a video walkthrough of two of the most memorable bugs you encountered during your project. For each bug, include: • What the bug was and how you discovered it • The debugging process you used to identify the root cause • How you fixed the bug • What you learned from the experience • The test cases you created to prevent this bug from reoccurring Link your video in your a5.pdf

Bug 1 Alignment algorithm worked in practice but failed on the testing set.

Bug1Video

Bug Description and Discovery The bug was a critical misalignment in our audio alignment module, which is responsible for syncing user and reference dance videos. While our initial onset-envelope cross-correlation algorithm passed our manual eye tests using random TikTok videos, running it against our testing dataset revealed a massive issue. We discovered an average offset(the time difference between where our algorithm predicts the videos sync and where they actually align) error of nearly 0.6 seconds, which is a significant failure since our downstream pose-grading algorithm requires frame-perfect alignment to function properly.

Debugging Process and Root Cause

We initially assumed the issue was a simple implementation error and spent time stepping through the codebase, but we found no bugs and realized the algorithm was working properly. The real breakthrough came when we graphed the error distribution rather than just looking at the averages. This revealed the root cause: the original algorithm was, very infrequently, unable to align audio files and returned extremely large alignment errors which skewed the average alignment results. When evaluating the algorithm's predictions of being within 1/60th of a second of perfect alignment, it was only within that range 93.7% of the time. The remaining errors were extreme outliers that skewed the average and would fail on video syncs.

Resolution

To fix this, we experimented with other approaches and ultimately replaced the original method with a modified Smith-Waterman algorithm. Because we are always comparing dance clips of the exact same length, we optimized it to only evaluate exact sequences rather than searching for an optimal path with gaps. This new algorithm dramatically improved our true metric: it successfully aligned the videos within 1/60th of a second 99.7% of the time on our testing data. This sub-frame accuracy completely resolved the downstream grading failures. This test can be seen in tests/alignment_and_segmentation/test_alignment_algo_accuracy.py.

Lessons Learned
This experience taught us the reality of engineering trade-offs. The Smith-Waterman algorithm uses dynamic programming, making it slower and more computationally expensive as clip

length increases. However, achieving that 99.7% accuracy rate is entirely worth the computational cost for our core product to work. It also underscored the necessity of rigorous, dataset-driven testing; if we had relied solely on manual testing with TikTok videos, this bug would have made it to production.

Preventative Testing Pipeline

To ensure this doesn't happen again, we built an automated testing pipeline that runs our algorithms against a curated dataset of known audio pairs. Crucially, instead of relying on average errors, which could easily hide extreme outliers if the errors are not as extreme as our first algorithm, the script specifically evaluates the percentage of alignments that fall outside our acceptable time range (1/60 of a second). If the system fails to maintain that sub-frame accuracy on at least 99.5% of the clips, it instantly fails the build.

Bug 2. Misalignment in OS - Windows: FFmpeg discovery / EBS pipeline blocked

What the bug was and how we discovered it

The bug was that the code was unable to run on the windows end, namely, it gets stuck and the EBS pipeline never finishes, blocking users from entering the main analysis page.
[Bug2Video]

The debugging process we used to identify the root cause

First we thoroughly checked if the running process is correct to confirm that this is an actual bug, not caused by improper procedure. After confirmation, we spotted the lack of .env.* file, but after that is pushed the bug remains. So we analyzed the backend EBS Error Log as the frontend has no issues:

```
14:51:46 [ebs-fastapi] INFO: Received video files — running EBS pipeline
(session=session-1774291906545-jpmxlp)
14:51:46 [ebs-fastapi] INFO: Extracting audio…
14:51:47 [ebs] INFO: Extracting audio via librosa/audioread:
C:\Users\zsq59\AppData\Local\Temp\ebs_ref_za6f5qgx.mp4
C:\Users\zsq59\csc490\TempoFlow\a2\pipelines\Audio data processing\ebs_segment.py:169:
UserWarning: PySoundFile failed. Trying audioread instead.
  audio, _ = librosa.load(video_path, sr=sr, mono=True)
C:\Users\zsq59\AppData\Local\Programs\Python\Python311\Lib\site-packages\librosa\core\audi
o.py:184: FutureWarning: librosa.core.audio.__audioread_load
        Deprecated as of librosa version 0.10.0.
        It will be removed in librosa version 1.0.
  y, sr_native = __audioread_load(path, offset, duration, dtype)
14:51:48 [ebs-fastapi] ERROR: Pipeline error
Traceback (most recent call last):
```

```
  File
"C:\Users\zsq59\AppData\Local\Programs\Python\Python311\Lib\site-packages\librosa\core\aud
io.py", line 176, in load
    y, sr_native = __soundfile_load(path, offset, duration, dtype)
                   ^^^
```

```
  File
"C:\Users\zsq59\AppData\Local\Programs\Python\Python311\Lib\site-packages\audioread\__init
__.py", line 132, in audio_open
    raise NoBackendError()
audioread.exceptions.NoBackendError
INFO:     127.0.0.1:59560 - "GET /api/status?session=session-1774291906545-jpmxlp
HTTP/1.1" 200 OK
INFO:     127.0.0.1:59560 - "GET /api/status?session=session-1774291906545-jpmxlp
HTTP/1.1" 200 OK
INFO:     127.0.0.1:59540 - "POST /api/process HTTP/1.1" 500 Internal Server Error
```

This log tells us that there is something wrong with the audioread_load function from librosa. Interestingly, we found that only one person experienced this bug, and we quickly realized the former development was produced on Mac only and never tested on a windows machine. Thus, this bug is likely due to an OS mismatch, namely, on Mac, audioread usually finds the system-installed FFmpeg or the native CoreAudio frameworks without any extra configuration. However this is not the case for Windows users, the lack of manually added C:\ffmpeg\bin to your System Environment Variables (PATH), audioread has no idea where to look, resulting in that NoBackendError.

How we fixed the bug

First, we verified the diagnosis by applying a temporary Windows-only fix (`FFMPEG_PATH = r"C:\ffmpeg\bin\ffmpeg.exe"`), which confirmed the diagnosis by demonstrating the problem was indeed the missing FFmpeg discovery. Then, we implemented a fix for the discovery by creating a cross-platform resolver module (`ebs_ffmpeg_paths.py`) to look for the binaries in a robust manner: (1) environment variables (`EBS_FFMPEG_PATH`, `EBS_FFPROBE_PATH`), (2) normal PATH search via `shutil.which`, and (3) typical Windows installation directories `%ProgramFiles%\ffmpeg\bin`, `%ProgramFiles(x86)%\ffmpeg\bin`, and `C:\ffmpeg\bin`. Finally, we incorporated this resolver module into the paths for both audio extraction and probing (`ebs_segment.py`, `ebs_fastapi_server.py`, and `ebs_server.py`). Now, the EBS pipeline functions correctly on macOS and Windows without the need for local paths. Video of fixed demo can be found here:

csc490a5bug2-fixed-vid.mp4

What we learned from the experience

This is a valuable experience, reminding us that OS differences can cause serious issues, and it is very important to test in both environments (not only macOS) to ensure our product runs. Within our team, we had two of our group members facing these issues. They were Windows users, and this bug actually took time and prevented them from working on the web-app, and instead led them to focus on the backend while the issue was being worked on.

The test cases we created to prevent this bug from recurring

`A5/tests/test_ffmpeg_paths.py` covers `resolve_ffmpeg_executable()` and `resolve_ffprobe_executable()` in `A5/src/ffmpeg_paths.py`: environment overrides (`EBS_FFMPEG_PATH`, `EBS_FFPROBE_PATH`), `shutil.which`, Windows install-directory

fallbacks, and default executable names when nothing else matches. Those tests check discovery logic quickly in CI without running full librosa loads on sample videos. Before a release we still do a short end-to-end run on a Windows machine with a typical install to confirm FFmpeg is actually picked up in practice.